Lesson 03 · Types, OOP & Generics

Classes, structs, records and enums · inheritance and interfaces · reified generics · C# 14 extension members and equality — for a Java/Kotlin architect

Contents

1 · Why the type system is where Java instincts break 2
1.1 · The numbers to leave this lesson with 2
2 · Five kinds of type — and where their bytes live 2
2.1 · The five kinds, side by side 3
2.2 · Where the bytes live — three arrays of 1,000 amounts 4
2.3 · Money — a value object as a readonly record struct 4
2.4 · Two copies you did not ask for — a struct read from a list, and default(Money) 4
2.5 · Bytes allocated for 1,000 amounts 5
2.6 · Methods the compiler emits for two properties 5
2.7 · Which kind of type? — the MotorQuote decisions 5
2.8 · Concept map — Java, Kotlin and TypeScript types → C# 5
3 · Members — properties, construction and the C# 14 field keyword 6
3.1 · Quote — an entity with required members and a guarded status 7
3.2 · The quote lifecycle the Status setter enforces 7
3.3 · Construction — Kotlin named arguments vs a C# object initializer 7
3.4 · A DI-friendly service — Kotlin constructor vs C# primary constructor 8
3.5 · Property forms — who can set the value, and when 8
4 · Inheritance and interfaces — methods are non-virtual by default 8
4.1 · The same hierarchy — Java's virtual default vs C#'s non-virtual default 9
4.2 · One object, three calls 9
4.3 · Which method runs? — rule.M() where rule is declared as the base type 9
4.4 · A default interface method on the rate table 10
4.5 · Static abstract members — a contract on the type 10
4.6 · Inheritance keywords in the four languages you will read 10
4.7 · The same rules in Visual Basic — what you will read in a legacy estate 11
5 · Generics — reified, constrained and variant 11
5.1 · Bytes to store 1,000 integers — reified vs boxed collections 12
5.2 · Parsing identifiers — Java's erasure workaround vs a reified type parameter 12
5.3 · Variance on the report contracts — Kotlin vs C# 12
5.4 · Generic math — one Sum for decimal and Money 13
5.5 · Constraints you will write and read 13
5.6 · The same constraint in Visual Basic — (Of T As ...) 13
6 · Extension members, equality and operators 13
6.1 · Behaviour for CoverageClass — Kotlin enum class vs a C# 14 extension block 14
6.2 · A classic extension method — the form LINQ and services.AddX() use 14
6.3 · Case-insensitive VIN equality — by hand, and as a record 15
6.4 · Boxes, overloaded == and a broken hash code 15
7 · Design in C# — a multi-format report framework, extended from VB 15
7.1 · The report framework — one C# contract, renderers in two languages 16
7.2 · The template-method base class 16
7.3 · The strategy selector — DI-friendly and case-insensitive 17
7.4 · The same CSV renderer in C# and in Visual Basic 17
7.5 · What Visual Basic does with C# records, operators and required members 18
7.6 · What the Visual Basic compiler says — every row asserted by VbCompilerTests 18
7.7 · Pinning the boundary — Visual Basic compiled inside an xUnit test 19
7.8 · Declare, consume or work around — Visual Basic on modern C# types 19
8 · Hands-on — build, test and run the lesson-03 samples 19
8.1 · Commands 20
8.2 · What you should see — the C# tour 20
8.3 · What you should see — Visual Basic on the C# types 21
8.4 · Code lines per sample project 21
8.5 · Test cases per test class 21

Page numbers are resolved from the rendered PDF. Code panels are cut from the compiled samples.

1 · Why the type system is where Java instincts break

C#'s classes, interfaces and generics look like Java's, and three decisions underneath them are different enough to ship bugs. Every C# type is a reference type or a value type, fixed at its declaration. Methods are non-virtual unless the base class says `virtual`. Generics are real types at run time. After lesson 02 (C# Language Essentials) you can read most C#; these three are why code that reads right can still behave wrong.

For an architect, the type system is the domain model. In a Kotlin service you chose between a `data class`, a `value class`, an open class and a `sealed` hierarchy. C# offers `class`, `struct`, `record`, `record struct` and `enum`, and the choice decides allocation, copying and equality. This lesson makes that choice for every MotorQuote concept — `Money`, `QuoteId`, `Vehicle`, `Driver`, `Quote` — and measures what each choice costs.

You already know OOP, SOLID and dependency injection, so the words go to the .NET shape. The lesson ends by rebuilding a pattern you have designed before — a multi-format report framework for PDF, Excel and JSON output — as a generic strategy over a template-method base class, then extends it from Visual Basic, the way a modernisation estate mixes the two languages.

Scope. Nullability, pattern matching and `decimal` belong to lesson 02 (C# Language Essentials); collections and LINQ to lesson 04 (Collections, LINQ & Functional C#); DI lifetimes and container registration to lesson 08 (ASP.NET Core Web APIs). Every feature here is C# 14, which shipped with .NET 10 in November 2025 • Verified (C# version history); C# 15 preview features are labelled as preview.

1.1 · The numbers to leave this lesson with

C# 14 · SHIPS WITH .NET 10	METHODS FOR TWO PROPERTIES	MONEY IN MEMORY	LESSON 03 TESTS	LESSON 03 SAMPLES
Nov 2025	**16 vs 3**	**24 bytes**	**80 cases**	**1,177 lines**
verified · C# version history (§1)	record vs class · reflection · measured	Unsafe.SizeOf on x64 · measured	xUnit facts + inline data · measured at build	C# + VB · 6 projects · measured at build

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured C# runtime

i Before you start

1. **Read first:** lesson 01 (The .NET Platform Map) for value types and the runtime, and lesson 02 (C# Language Essentials) for syntax, nullability, pattern matching and `decimal`.
2. **You will build six projects:** `L03.Domain` (the MotorQuote types), `L03.Reports` (a generic report framework), two xUnit test projects, `L03.TypesTour` (a console that prints every measurement quoted here) and `L03.VbInterop` (Visual Basic using the C# types and extending the framework).
3. **Premiums follow the curriculum's canonical tariff — illustrative, not a real tariff.** The same rates (Class 1 2.1% of the sum insured, Class 2+ 1.2%, Class 3+ 0.9%, Class 3 0.4%), the same loadings (young driver +20%, one claim +10%, two +25%, commercial use +25% or +35% over 3,000 cc), the same no-claim ladder (0/20/25/30/40/50% by claim-free licence year) and the same stamp duty (0.4% of net) and VAT (7% of net plus duty) run through every lesson, so a premium you compute here matches the one you persist in lesson 09 (Data Access with EF Core) and test in lesson 10 (Testing, TDD & 100% Coverage). Three or more claims in five years declines the quote: this lesson teaches types and exceptions, so `ClaimsLoading` throws `QuoteDeclinedException` (the data-shaped lessons record `QuoteStatus.Declined` instead).
4. Facts that change with releases are marked • Verified and link to Microsoft Learn; judgements are marked • Estimate; figures computed from the samples are marked • Measured. Byte counts come from one run of `L03.TypesTour` in a 64-bit process on .NET 10.0.12 — object sizes depend on the process architecture; the ratios are the point.

2 · Five kinds of type — and where their bytes live

C# makes you choose reference or value semantics when you declare a type, not when you use it. `class` and `record` are reference types: allocated on the heap, shared by reference, `null` by default. `struct`, `record struct` and `enum` are value types: stored inline in their variable, field or array slot, copied on every assignment, and all-zero by default. Java gives you that split only for its primitives, and Kotlin's `value class` wraps exactly one property. In C# any type you write can be a value type (structure types).

The difference is measurable. A thousand `Money` values in an array are one allocation of 24,024 bytes. The same thousand amounts as class instances cost 48,024 bytes and 1,000 heap objects for the garbage collector to trace. Put the structs into an `object[]` and you are back at 48,024: boxing copies each value into its own heap object (boxing and unboxing).

Records are the compiler writing the members you used to generate. A two-property `record` compiles to 16 methods — value `Equals`, `GetHashCode`, `==`, `ToString`, `Deconstruct` and the clone behind `with` — against 3 for the equivalent class (records). It is Kotlin's `data class`, offered as a reference type and as a value type.

Microsoft's struct guideline is strict, and `Money` breaks one rule on purpose. The guideline says to avoid a struct unless the type is one logical value, under 16 bytes, immutable and rarely boxed (choosing between class and struct). `Money` is 24 bytes — a 16-byte `decimal` plus an 8-byte string reference — but it is immutable, compared constantly and stored in arrays of premium lines. That trade is a judgement (• Estimate): measure before you copy it.

2.1 · The five kinds, side by side

type-system view

1 class — a reference type with identity C# • Different

Allocated on the heap, compared by reference, null by default.

JVM	Any Java or Kotlin class — but open for inheritance until you write `sealed`
EQUALITY	Reference, unless the type overloads `==` and `Equals`
USE FOR	Entities with identity and a lifecycle (`Quote`) and services (`PremiumCalculator`)
TRAP	Kotlin classes are final by default; C# classes are not

2 struct — a value type runtime • Different

Stored inline, copied on every assignment, never null.

JVM	Nothing user-defined in mainstream Java; the nearest thing is a primitive
EQUALITY	`Equals` compares fields through reflection; no `==` until you declare one
USE FOR	Small immutable values on hot paths; interop buffers
TRAP	A mutable struct read from a `List<T>` is a copy — changes to it vanish (panel 2.4)

2.1 · The five kinds (continued)

type-system view

3 record — a class whose data members the compiler writes C# • Same idea

Value equality, ToString, Deconstruct and non-destructive copy with `with`.

JVM	Kotlin `data class` · Java `record` (C# records can inherit)
GENERATED	16 methods for two properties (chart 2.6)
USE FOR	Requests, messages and report rows: data without identity
TRAP	A `List<T>` property compares by reference, not by contents

4 record struct — a value type with record members C# • Different

`readonly record struct` is the immutable form, and the usual choice.

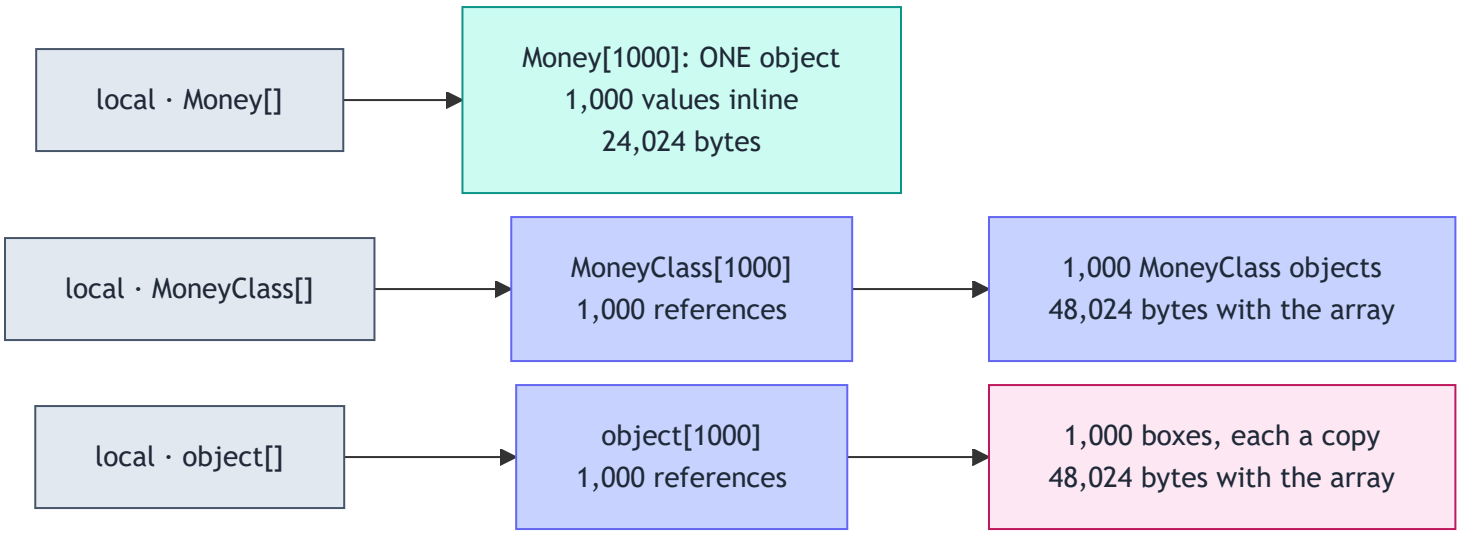
JVM	Kotlin `value class`, without the one-property limit
SIZE	`Money` = 24 bytes; `QuoteId` = 4 bytes (measured)
USE FOR	Value objects: `Money`, `QuoteId`, `PolicyNumber`
TRAP	`default(Money)` runs no constructor: its `Currency` is null

5 enum — named integers runtime • Trap

Named constants over an integral type, `int` unless you say otherwise.

JVM	Java and Kotlin enums are classes with fields and methods; C# enums are not
SIZE	`CoverageClass` = 4 bytes (measured)
BEHAVIOUR	Add members with a C# 14 extension block (§6)
TRAP	Any `int` converts: `(CoverageClass)42` is a legal value

2.2 · Where the bytes live — three arrays of 1,000 amounts



slate = variable · teal = values inline · indigo = heap objects · rose = boxes copied out of a struct · bytes measured (chart 2.5)

2.3 · Money — a value object as a readonly record struct

C# — samples/L03.Domain/Money.cs · #region money

```
/// <summary>An amount in one currency: a value object, not an entity.</summary>
public readonly record struct Money(decimal Amount, string Currency)
    : IAdditionOperators<Money, Money, Money>
{
    public static Money Thb(decimal amount) => new(amount, "THB");

    public static Money operator +(Money left, Money right) =>
        left.Currency == right.Currency
            ? new(left.Amount + right.Amount, left.Currency)
            : throw new InvalidOperationException(
                $"cannot add {right.Currency} to {left.Currency}");

    public static Money operator *(Money money, decimal factor) =>
        money with
        {
            Amount = decimal.Round(money.Amount * factor, 2,
                MidpointRounding.AwayFromZero),
        };

    public override string ToString() =>
        Amount.ToString("N2", CultureInfo.InvariantCulture) + " " + Currency;
}
```

Read the declaration line first. `readonly` makes every field immutable, `record` generates value equality, `==`, `ToString` and `Deconstruct` (`with` works on any struct), `struct` stores it inline. Implementing `IAdditionOperators<Money, Money, Money>` declares `+` as a static interface member — that is what lets the generic `Totals.Sum` in §5 add money. Mixed currencies throw: a THB premium plus a USD fee is a bug.

2.4 · Two copies you did not ask for — a struct read from a list, and default(Money)

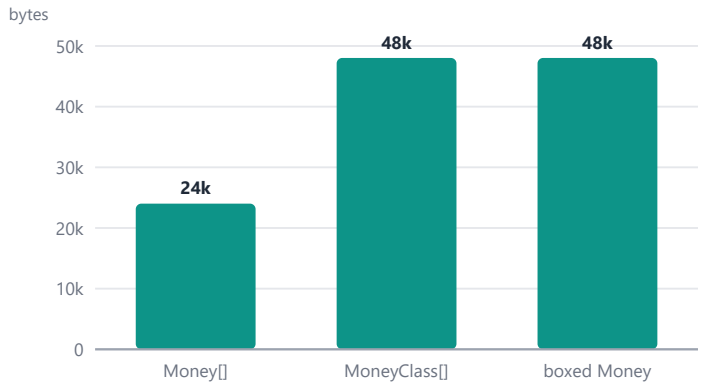
C# — samples/L03.TypesTour/Program.cs · #region copy-trap

```
var tallies = new List<Tally> { new() };
var copy = tallies[0];        // Tally is a struct: this is a COPY
copy.Add();
Console.WriteLine($"copy       copy.Count={copy.Count} list[0].Count={tallies[0].Count}");

var blank = default(Money);   // no constructor ran
Console.WriteLine($"default    default(Money).Currency is null: {blank.Currency is null}");
```

Value semantics bite where Java has no value types. `Tally` is a small mutable struct. The list indexer returns a copy, so `copy.Add()` changes only the copy: the tour prints `copy.Count=1 list[0].Count=0`. An array element is a variable, so the same call on a `Tally[]` changes it in place (`CopySemanticsTests` asserts both). `default(Money)` runs no constructor, so the non-nullable `Currency` is null (`default(Money).Currency is null: True`). Both are reasons to keep structs `readonly`.

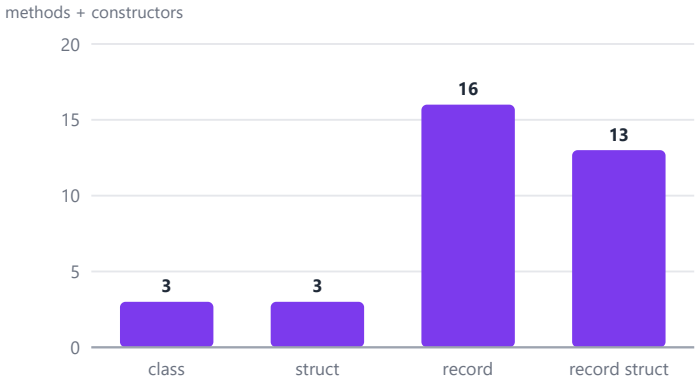
2.5 · Bytes allocated for 1,000 amounts



GC.GetAllocatedBytesForCurrentThread around each loop · L03.TypesTour · measured

Value types halve the bytes and remove 1,000 objects. The class array holds 1,000 references to 1,000 separate objects; boxing turns the struct array back into exactly that shape.

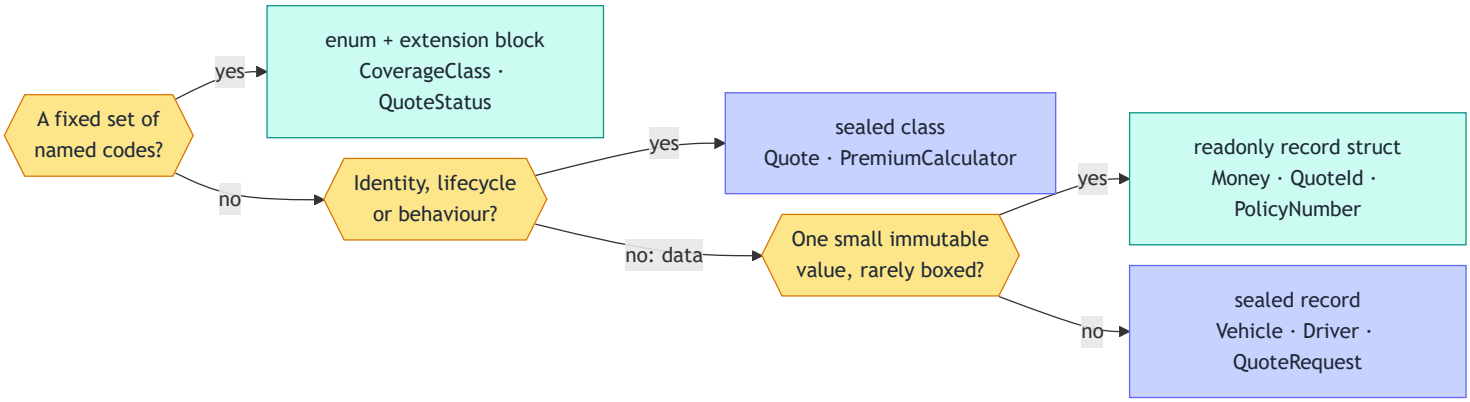
2.6 · Methods the compiler emits for two properties



declared methods and constructors found by reflection · L03.TypesTour · measured

A record is a class plus 13 generated members. The record class needs three that a record struct does not: a copy constructor, the compiler-named clone method behind `with` (virtual unless the record is sealed) and `EqualityContract`, which makes equality compare the run-time type.

2.7 · Which kind of type? — the MotorQuote decisions



amber = question · indigo = reference type · teal = value type · each answer names the MotorQuote types that took that path

2.8 · Concept map — Java, Kotlin and TypeScript types → C#

You already know	In .NET	Kind	Watch for
Java class · Kotlin `open class`	`class`	• Different	Open for inheritance by default (Kotlin: final); mark leaf classes `sealed`
Java / Kotlin `final` class	`sealed`	• Renamed	Not Kotlin's `sealed`: nothing about exhaustiveness
Kotlin `sealed class` + exhaustive `when`	no C# 14 equivalent; `closed` classes are in the C# 15 preview	• Trap	A C# 14 switch over a class hierarchy needs a default arm
Kotlin `data class` · Java `record`	`record`	• Same idea	`with` = `copy()`; equality also compares the run-time type
Kotlin `value class` · Java primitives	`readonly record struct`	• Different	Many fields allowed; copied on assignment; `default` skips the constructor
Java enum · Kotlin `enum class`	`enum` (named integers)	• Trap	No fields or methods; `(CoverageClass)42` is legal
getters and setters · Lombok builders	properties · `init` · `required`	• Renamed	Object initializers replace most builders

You already know	In .NET	Kind	Watch for
Kotlin `class X(val a: A)`	primary constructor `class X(A a)`	• Trap	Parameters, not properties — only records generate properties
Kotlin `field` in a custom setter	C# 14 `field` keyword	• Same idea	Guard a setter without declaring a backing field
Java methods (virtual by default)	non-virtual unless `virtual`	• Trap	A same-named method without `override` hides the base one (CS0114 · CS0108)
Kotlin `open fun` · `override`	`virtual` · `override`	• Trap	Leaving out `override` still compiles: the method hides the base one with warning CS0114, where Kotlin rejects it. Warnings as errors gives you Kotlin's rule
Java interface `default` method	default interface method	• Different	Callable only through the interface type, not the class
Kotlin `companion object` · Java `static`	`static` · `static abstract` in interfaces	• Different	Generic code can call `T.Parse` on a type parameter
Type erasure · `Class<T>` tokens	reified generics	• Different	`typeof(T)`, `default(T)`, statics per closed type, no boxing in `List<int>`
Java `? extends` · `? super`	`out` · `in` on interfaces and delegates	• Different	Declaration-site only; `List<T>` stays invariant
Kotlin `out` · `in`	`out` · `in`	• Same idea	Value-type arguments are always invariant
Kotlin extension functions and properties	C# 14 `extension` blocks	• Same idea	Declared in a static class; no state; VB cannot use the property syntax
Kotlin `==` (calls `equals`)	`==`: reference unless overloaded	• Trap	Records and `string` overload it; plain classes and boxes do not
TypeScript structural typing	nominal typing	• Different	Matching shape is not enough — the type must declare the interface

3 · Members — properties, construction and the C# 14 field keyword

Properties are the unit of state in C#, and the compiler writes most of them for you. `{ get; set; }` is a Kotlin `var`; `{ get; init; }` (C# 9) can be set during construction and never again; `required` (C# 11) makes a missing member a compile error (required members); and the C# 14 `field` keyword lets a setter validate without a hand-declared backing field (the field keyword). Hand-written getters and setters are a Java habit to drop.

Construction is an object initializer checked by the compiler, not a builder. `new Quote { Id = …, Request = …, Premium = … }` fails to compile if any `required` member is left out, so the Lombok `@Builder` and the Kotlin named-argument constructor both collapse into one checked expression. Copies of records use `with`.

A primary constructor on a class is a parameter list, not a property list. Kotlin's `class PremiumCalculator(private val rates: RateTable)` declares a property. C#'s `class PremiumCalculator(IRateTable rates)` (C# 12) declares a parameter that is in scope for the whole class, captured into a hidden field when a method uses it — and that field is assignable, not `readonly` (primary constructors). Only `record` turns positional parameters into properties. It is still the idiomatic shape for a DI-constructed service.

Statics sit on the type, as in Java. Kotlin's `companion object` becomes `static` members; `const` versus `static readonly` is covered in lesson 02 (C# Language Essentials).

3.1 · Quote — an entity with required members and a guarded status

C# — samples/L03.Domain/Quote.cs · #region quote-entity

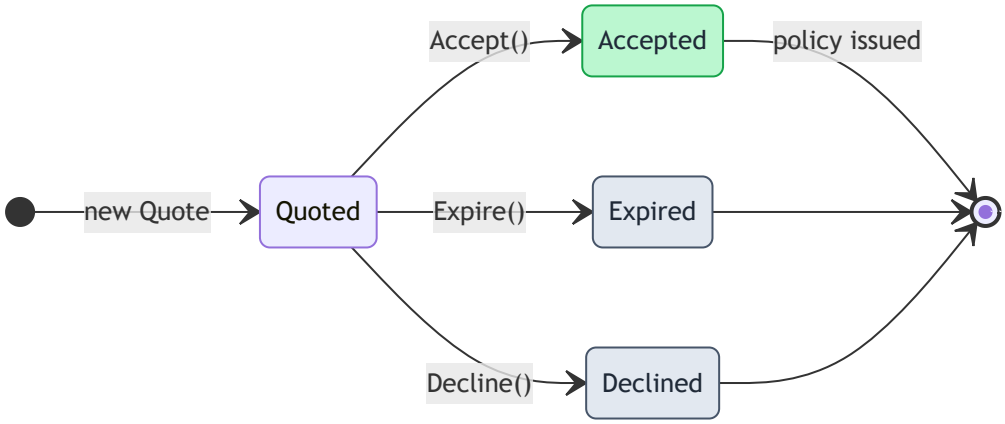
```
public required QuoteId Id { get; init; }
public required QuoteRequest Request { get; init; }
public required Premium Premium { get; init; }
public DateOnly ValidUntil { get; init; }

// C# 14 `field`: guard a setter without declaring a backing field
public QuoteStatus Status
{
    get;
    private set => field = CanMove(field, value)
        ? value
        : throw new InvalidOperationException($"{field} -> {value}");
} = QuoteStatus.Quoted;

public Policy Accept(PolicyNumber number)
{
    Status = QuoteStatus.Accepted;
    return new Policy(number, Id,
        Request.StartDate, Request.StartDate.AddYears(1));
}
```

Three member forms in twenty lines. The three `required` properties must appear in every `new Quote { … }`. `ValidUntil` is `init`: optional, then frozen. `Status` uses `field` so the setter can check the move against the lifecycle in diagram 3.2; the initializer `= QuoteStatus.Quoted` writes the backing field directly and skips the check, which is why a new quote starts at `Quoted`. `Quote` is a `class`, not a record: two quotes with equal data are still two quotes.

3.2 · The quote lifecycle the Status setter enforces



green = issues a policy · slate = terminal · any move not drawn throws InvalidOperationException from the field-backed setter · QuoteStatus also has Draft, which this entity never uses: it starts at Quoted

3.3 · Construction — Kotlin named arguments vs a C# object initializer

Kotlin — for comparison — the idea you already know

```
// named arguments: a parameter with no
// default must be passed, or it won't compile
fun quote(number: Int, req: QuoteRequest) =
    Quote(
        id = QuoteId(number),
        request = req,
        premium = calculator().calculate(req),
        validUntil = req.startDate, // has a default
    )
```

C# — samples/L03.Domain/Samples.cs · #region object-initializer

```
// leave out a required member and this does not compile
public static Quote Quote(int number, QuoteRequest req) =>
    new Quote
    {
        Id = new QuoteId(number),
        Request = req,
        Premium = Calculator().Calculate(req),
        ValidUntil = req.StartDate, // init, but optional
    };
```

• **Renamed** — **the check moves from the constructor to the members.** Kotlin enforces completeness through constructor parameters without defaults; C# marks the members `required` and checks every `new Quote { … }`. Leave out `Premium` and the build fails with CS9035. The initializer runs after the constructor, which is why `init` accessors accept its assignments.

3.4 · A DI-friendly service — Kotlin constructor vs C# primary constructor

Kotlin — for comparison — the idea you already know

```kotlin
class PremiumCalculator(
    private val rates: RateTable,
    rules: List<RatingRule>,
) {
    // `val` makes a read-only property; a plain
    // parameter is visible to initializers only
    private val rules = rules.toList()

    // loadings add up, a discount multiplies
    fun calculate(req: QuoteRequest): Premium {
        var load = ZERO; var disc = ONE
        for (r in rules)
            if (r.isDiscount) disc *= r.factor(req)
            else load += r.factor(req) - ONE
        return Premium(rates.basePremium(req)
            * ((ONE + load) * disc))
    }
}
```

C# — samples/L03.Domain/Rating.cs · #region primary-ctor

```csharp
// C# 12 primary constructor: the DI shape, no boilerplate
public sealed class PremiumCalculator(
    IRateTable rates, IEnumerable<RatingRule> rules)
{
    // `rules` is a parameter, not a property, not readonly:
    // copy it once: nothing can re-assign or re-enumerate it
    private readonly RatingRule[] _rules = [.. rules];

    public Premium Calculate(QuoteRequest request)
    {
        var loadings = 0m;
        var discount = 1.00m;
        foreach (var rule in _rules)
            if (rule.IsDiscount)
                discount *= rule.Factor(request);
            else
                loadings += rule.Factor(request) - 1.00m;

        // one rounding, on the tariff's own composition
        var net = rates.BasePremium(request)   // captured
                  * ((1.00m + loadings) * discount);
        return new Premium(net);
    }
}
```

• Trap **for Kotlin hands.** There is no `val` in a C# class's primary constructor. `rates` is used inside a method, so the compiler stores it in a hidden field that your own code could reassign; `rules` is used only in a field initializer, so it is never stored at all. Copying `rules` into a `readonly` array is the C# way to say *this dependency does not change*. Both sides compose the canonical tariff the same way — every loading is *added* to one factor, the no-claim discount multiplies, and the single rounding happens in `Money`'s `*` operator (panel 2.3).

3.5 · Property forms — who can set the value, and when

Form	Declaration	Who can set it	From your stack
Auto property	`{ get; set; }`	Anyone, at any time	Kotlin `var`
Get-only	`{ get; }`	The constructor only	Kotlin `val` · Java final field
Init-only · C# 9	`{ get; init; }`	Constructor or object initializer, then never	Kotlin `val` set by named argument
Required · C# 11	`required … { get; init; }`	Must be set in the initializer — a compile error otherwise	Kotlin constructor parameter without a default
Field-backed · C# 14	`set => field = …`	As the accessors allow; `field` is the compiler's backing field	Kotlin `field` in a custom setter
Computed	`Money Total => …;`	Nobody — evaluated on every read	Kotlin `val total get() = …`

4 · Inheritance and interfaces — methods are non-virtual by default

In C#, a method can be overridden only if its base class declares it `virtual` or `abstract`. In Java every instance method is virtual unless it is final, private or static; in Kotlin methods are final until marked open. So for methods C# is Kotlin's • Same idea, Java's • Trap — but C# *classes* are open by default, like Java and unlike Kotlin (versioning with override and new).

A derived method with the same signature and no `override` does not override — it hides. The compiler warns — CS0114 when the base member is virtual, CS0108 when it is not — this repository turns the warning into an error, and `new` silences it by declaring the hiding intentional. Which method runs then depends on the *declared* type of the variable, not the object. You meet `new` in legacy code when a base library later added a member with a name a subclass already used; in new code it is almost always a mistake.

Interfaces gained default methods and static abstract members, but never state. A default interface method (C# 8) is reachable only through an interface-typed reference — `new StandardRateTable().BasePremium(…)` does not compile. A `static abstract` member (C# 11) is a contract on the type rather than the instance, resolved at compile time; it is what generic math and `T.Parse` in §5 are built on (interface members).

C# 14 has no closed hierarchies. `sealed` means Java's `final`, not Kotlin's `sealed`, so a `switch` over subclasses cannot be proven exhaustive. The C# 15 preview adds a `closed` modifier for classes and union types for exactly this (What's new in C# 15

(preview), checked 2026-09-20) • Verified ; until .NET 11 ships, keep a default arm. Enums are not covered: an enum switch keeps its default arm (§6).

4.1 · The same hierarchy — Java's virtual default vs C#'s non-virtual default

Java — for comparison — the habit you bring

```java
abstract class RatingRule {
    abstract String name();

    // overridable, like every method below
    boolean isDiscount() { return false; }

    BigDecimal factor(QuoteRequest r) { return ONE; }

    // every instance method is virtual in Java
    String describe(QuoteRequest r) {
        return name() + " x" + factor(r);
    }
}

final class YoungDriverLoading extends RatingRule {
    String name() { return "young driver"; }

    @Override
    BigDecimal factor(QuoteRequest r) { /* 1.20 */ }

    // silently OVERRIDES describe(): @Override
    // is optional, the behaviour is not
    String describe(QuoteRequest r) {
        return name() + " (hidden copy)";
    }
}
```

C# — samples/L03.Domain/Rating.cs · #region dispatch

```csharp
public abstract class RatingRule
{
    public abstract string Name { get; }

    // virtual too: loadings add up, a discount multiplies
    public virtual bool IsDiscount => false;

    // virtual: a derived class MAY override it
    public virtual decimal Factor(QuoteRequest r) => 1.00m;

    // not virtual (the C# default): it can only be hidden
    public string Describe(QuoteRequest r) =>
        Invariant($"{Name} x{Factor(r)}");
}

public sealed class YoungDriverLoading : RatingRule
{
    public override string Name => "young driver";

    public override decimal Factor(QuoteRequest r) =>
        r.Driver.AgeOn(r.StartDate) < 25 ? 1.20m : 1.00m;

    // `new` HIDES the base method; it does not override it
    public new string Describe(QuoteRequest r) =>
        $"{Name} (hidden copy)";
}
```

Identical shape, different call. In Java, `rule.describe(q)` on a `RatingRule` variable runs the subclass method. In C#, `Describe` is not virtual, so the subclass can only hide it with `new` — and a `RatingRule`-typed variable still calls the base version. `Factor` *is* virtual, so the base `Describe` picks up the derived 1.20.

4.2 · One object, three calls

C# — samples/L03.TypesTour/Program.cs · #region dispatch-demo

```csharp
RatingRule asBase = new YoungDriverLoading();
var asDerived = (YoungDriverLoading)asBase;

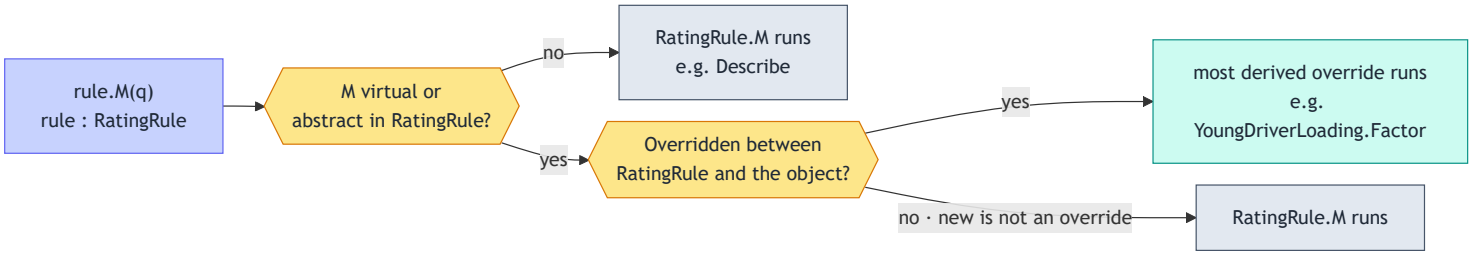
Console.WriteLine(asBase.Factor(young));      // virtual
Console.WriteLine(asBase.Describe(young));    // static type
Console.WriteLine(asDerived.Describe(young)); // hiding one
```

Output — L03.TypesTour — captured on the build machine

```
1.20
young driver x1.20
young driver (hidden copy)
```

One object, two answers to `Describe`. The virtual `Factor` follows the object; the non-virtual `Describe` follows the variable's declared type. `DispatchTests` asserts all three lines, so a refactoring that adds `virtual` fails a test instead of silently changing a premium description.

4.3 · Which method runs? — rule.M() where rule is declared as the base type



amber = question the compiler or runtime asks · slate = the declared type decides · teal = the object's run-time type decides

4.4 · A default interface method on the rate table

C# — samples/L03.Domain/Rating.cs · #region interfaces

```
public interface IRateTable
{
    decimal RateFor(CoverageClass coverage);

    // C# 8 default interface method: a class need not implement it, but
    // it does not inherit it: call it through an IRateTable reference
    Money BasePremium(QuoteRequest request) =>
        request.Vehicle.SumInsured * RateFor(request.Coverage);
}

/// <summary>The curriculum tariff: illustrative rates, not a real tariff.</summary>
public sealed class StandardRateTable : IRateTable
{
    public decimal RateFor(CoverageClass coverage) => coverage switch
    {
        CoverageClass.Class1 => 0.021m,
        CoverageClass.Class2Plus => 0.012m,
        CoverageClass.Class3Plus => 0.009m,
        CoverageClass.Class3 => 0.004m,
        _ => throw new ArgumentOutOfRangeException(nameof(coverage)),
    };
}
```

● **Different from Java defaults.** `StandardRateTable` implements only `RateFor`. It does not inherit `BasePremium`: the interface supplies the body, so only an `IRateTable`-typed reference can call it (a Java class inherits its default methods). `PremiumCalculator` takes an `IRateTable`, so the call compiles; `L03.TypesTour` prints `9,450.00 THB` (450,000 × 2.1%, the canonical Class 1 rate), and `DispatchTests` asserts the class has no such member. A Visual Basic class implementing this interface must still write `BasePremium` itself (BC30149, table 7.6).

4.5 · Static abstract members — a contract on the type

C# — samples/L03.Domain/Identifiers.cs · #region static-abstract

```
// C# 11 static abstract members: a contract on the TYPE, not the instance
public interface IIdentifier<TSelf> where TSelf : IIdentifier<TSelf>
{
    static abstract string Prefix { get; }
    static abstract TSelf Parse(string text);
}

public readonly record struct QuoteId(int Value) : IIdentifier<QuoteId>
{
    public static string Prefix => "Q-";

    public static QuoteId Parse(string text) =>
        new(int.Parse(text[Prefix.Length..], CultureInfo.InvariantCulture));

    public override string ToString() => $"{Prefix}{Value:D6}";
}
```

An interface that describes the type, not the instance. Every `IIdentifier<TSelf>` must offer a static `Prefix` and a static `Parse`. Nothing can call them through an instance; generic code calls them through the type parameter, as §5 shows. The self-referencing constraint `where TSelf : IIdentifier<TSelf>` is the same F-bounded shape as Java's `<T extends Comparable<T>>` bound or `Enum<E extends Enum<E>>`.

4.6 · Inheritance keywords in the four languages you will read

Meaning	Java	Kotlin	C#	Visual Basic
Can be overridden	(default)	`open`	`virtual`	`Overridable`
Overrides a base member	`@Override` (optional)	`override`	`override`	`Overrides`
Must be overridden	`abstract`	`abstract`	`abstract`	`MustOverride`
Abstract class	`abstract class`	`abstract class`	`abstract class`	`MustInherit Class`
Cannot be inherited	`final class`	(default)	`sealed`	`NotInheritable`
Stops further overriding	`final` method	`final override`	`sealed override`	`NotOverridable Overrides`
Hides a base member	—	—	`new`	`Shadows` (hides every overload of the name)

Meaning	Java	Kotlin	C#	Visual Basic
Inherit · implement	`extends` · `implements`	`:`	`:`	`Inherits` · `Implements`
Call the base version	`super.m()`	`super.m()`	`base.M()`	`MyBase.M()`

4.7 · The same rules in Visual Basic — what you will read in a legacy estate

Visual Basic — samples/L03.VbInterop/Program.vb · #region vb-keywords

```vb
' panel 4.1's C# hierarchy, keyword for keyword (L03.VbInterop.RatingRule)
Public MustInherit Class RatingRule                                  ' abstract
    Public MustOverride ReadOnly Property Name As String

    Public Overridable Function Factor(r As QuoteRequest) As Decimal  ' virtual
        Return 1.00D
    End Function

    Public Function Describe(r As QuoteRequest) As String             ' not virtual
        Return $"{Name} x{Factor(r)}"
    End Function
End Class

Public NotInheritable Class YoungDriverLoading                       ' sealed
    Inherits RatingRule

    Public Overrides ReadOnly Property Name As String = "young driver"

    Public Overrides Function Factor(r As QuoteRequest) As Decimal    ' override
        Return If(r.Driver.AgeOn(r.StartDate) < 25, 1.20D, 1.00D)
    End Function

    Public Shadows Function Describe(r As QuoteRequest) As String     ' new
        Return $"{Name} (hidden copy)"
    End Function
End Class
```

• Renamed **keyword for keyword, same dispatch for this single method.** This is panel 4.1's C# hierarchy member for member, declared in the VB project. `MustInherit` and `MustOverride` are `abstract`, `Overridable` is `virtual`, `NotInheritable` is `sealed` and `Shadows` is `new` — with one difference that matters as soon as a name is overloaded: `Shadows` hides *every* overload of the name, where C# `new` hides only the matching signature (lesson 06 (VB.NET for Legacy Estates)). A VB method is non-virtual by default too, so `L03.VbInterop` prints `1.20 | young driver x1.20 | young driver (hidden copy)` (panel 8.3) — the same three answers as panel 4.2.

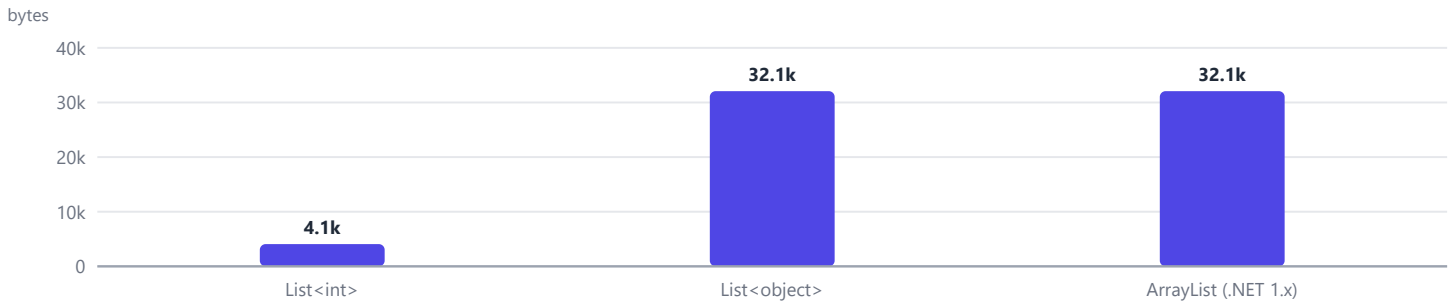
5 · Generics — reified, constrained and variant

C# generics survive compilation. `List<int>` and `List<string>` are different run-time types, and `List<int>` stores unboxed integers: 1,000 of them cost 4,056 bytes, against 32,056 for `List<object>` or the pre-generics `ArrayList` you still find in .NET Framework code (chart 5.1). Inside a generic method `typeof(T)` and `default(T)` always work, `new T()` works under a `where T : new()` constraint, and each closed type gets its own static fields. The Java workarounds for erasure — class tokens, `TypeReference` subclasses, `Supplier<T>` factories — have nothing to solve.

Constraints are where the design lives. `where T : IIdentifier<T>` lets `Ids.ParseAll<T>` call `T.Parse`, a static member — something no JVM generic can express. The same mechanism powers generic math: since .NET 7 and C# 11, operators are static abstract members of interfaces such as `IAdditionOperators` and `INumber<T>`, so one `Sum<T>` adds `decimal` and `Money` (generic math).

Variance is declared on the type, and only interfaces and delegates can have it. `in` and `out` mean exactly what they mean in Kotlin (• Same idea). Java's use-site wildcards have no C# form (• Different): `List<T>` is a class and stays invariant, and variance never applies to value-type arguments, so an `IEnumerable<int>` is not an `IEnumerable<object>` (covariance and contravariance in generics).

5.1 · Bytes to store 1,000 integers — reified vs boxed collections



GC.GetAllocatedBytesForCurrentThread, capacity preset to 1,000 · L03.TypesTour · measured

An eighth of the memory, and zero objects per element. `List<int>` is one list plus one `int[1000]`. `List<object>` — the shape every JVM `List<Integer>` has — and the .NET 1.x `ArrayList` add a 24-byte box per integer. When you port an `ArrayList`, the generic version is also the faster one.

5.2 · Parsing identifiers — Java's erasure workaround vs a reified type parameter

Java — for comparison — the erasure workaround

```java
// erasure: T does not exist at run time, so the
// caller passes a class token and a parser
static <T> List<T> parseAll(Class<T> type,
        Function<String, T> parse, String... texts) {
    var ids = new ArrayList<T>(texts.length);
    for (var text : texts) {
        ids.add(parse.apply(text));
    }
    return ids;
}

// T.parse(text), new T(), new T[n]: do not compile
// List<Integer> holds one boxed Integer per element
var ids = parseAll(QuoteId.class, QuoteId::parse,
        "Q-000042", "Q-000043");
```

C# — samples/L03.Domain/Identifiers.cs · #region reified

```csharp
public static class Ids
{
    // T is a real type at run time: no Class<T> token needed
    public static List<T> ParseAll<T>(params string[] texts)
        where T : IIdentifier<T>
    {
        var ids = new List<T>(texts.Length);
        foreach (var text in texts)
            ids.Add(T.Parse(text)); // a static call on T
        return ids;
    }

    public static string Describe<T>()
        where T : IIdentifier<T> =>
        $"{typeof(T).Name} {T.Prefix} {default(T)}";
}
```

The type argument carries the behaviour. `Ids.ParseAll<QuoteId>("Q-000042", "Q-000043")` needs no token and no lambda, because `IIdentifier<T>` promises a static `Parse`. `Describe<QuoteId>()` prints `QuoteId Q- Q-000000`: the real type name and a real `default(T)`. In the same run `Counter<QuoteId>.Hits` is 2 while `Counter<PolicyNumber>.Hits` is 1 — one static field per closed generic type.

5.3 · Variance on the report contracts — Kotlin vs C#

Kotlin — for comparison — the idea you already know

```kotlin
// declaration-site variance: the same keywords
interface ReportRenderer<in R : Report> {
    val format: String
    fun render(report: R): String
}

interface ReportSource<out R : Report> {
    fun load(): R
}

val card: ReportRenderer<QuoteReport> =
    SummaryCardRenderer()

// Java writes the same idea at every use site:
// void renderAll(ReportRenderer<? super QuoteReport> r)
// ReportSource<? extends Report> s = quoteSource;
```

C# — samples/L03.Reports/Contracts.cs · #region variance

```csharp
// `in` = contravariant: a renderer for ANY Report can
// stand in where a QuoteReport renderer is expected
public interface IReportRenderer<in TReport>
    where TReport : Report
{
    string Format { get; }
    string Render(TReport report);
}

// `out` = covariant: a source of QuoteReports
// is also a source of Reports
public interface IReportSource<out TReport>
    where TReport : Report
{
    TReport Load();
}
```

• Same idea **keywords, one C# restriction.** `SummaryCardRenderer` renders any `Report`, so it can be used where an `IReportRenderer<QuoteReport>` is expected — `VarianceTests` proves both directions. The compiler rejects `in` or `out` on a class, and a `List<QuoteReport>` is never a `List<Report>`.

5.4 · Generic math — one Sum for decimal and Money

C# — samples/L03.Domain/Totals.cs · #region generic-math

```
public static class Totals
{
    // one algorithm for int, decimal and Money: `+` is found per
    // type argument through a static abstract operator (.NET 7+)
    public static T Sum<T>(IEnumerable<T> items, T seed)
        where T : IAdditionOperators<T, T, T>
    {
        var total = seed;
        foreach (var item in items)
            total += item;
        return total;
    }
}
```

The constraint names an operator. `IAdditionOperators<T, T, T>` declares `static abstract T operator +`, so `total += item` compiles for any `T` that implements it: `decimal` does since .NET 7, and `Money` does because panel 2.3 declares it. `L03.TypesTour` prints `Sum<decimal>=6.50` and `Sum<Money>=1,500.00 THB`.

5.5 · Constraints you will write and read

Constraint	Means	Nearest Java / Kotlin	In the samples
`where T : class` · `struct`	a reference type · a non-nullable value type	no value-type generics on the JVM	—
`where T : notnull`	any non-nullable type	Kotlin `T : Any`	—
`where T : new()`	has a public parameterless constructor, so `new T()` compiles	a `Supplier<T>` parameter	—
`where T : Report`	derives from a class	`T extends Report`	`IReportRenderer<in TReport>`
`where T : IIdentifier<T>`	implements an interface — here one with static abstract members	F-bounded `T extends Comparable<T>`; no static calls	`Ids.ParseAll<T>`
`where T : IAdditionOperators<T,T,T>`	+ is defined for T	not expressible	`Totals.Sum<T>`
`where T : unmanaged`	a value type with no references inside	not expressible	—
`allows ref struct` · C# 13	T may be a stack-only type such as `Span<T>`	not applicable	—

5.6 · The same constraint in Visual Basic — (Of T As ...)

Visual Basic — samples/L03.VbInterop/Program.vb · #region vb-generic

```
' (Of T As constraint) is C#'s <T> where T : constraint
Function ParseBoth(Of T As IIdentifier(Of T))(
        first As String, second As String) As List(Of T)
    Return Ids.ParseAll(Of T)(first, second)
End Function
```

• Renamed **syntax, same reified generic.** `(Of T As IIdentifier(Of T))` is C#'s `<T> where T : IIdentifier<T>`; several constraints go in braces, `(Of T As {Class, New})`. VB can declare and call this method, but it cannot write `T.Parse` on its own type parameter (BC32098, table 7.6), so it hands the static call to the C# `Ids.ParseAll(Of T)`. `L03.VbInterop` prints `Q-000042, Q-000043`.

6 · Extension members, equality and operators

C# 14 extension blocks are Kotlin extensions with a different wrapper. Inside a static class, `extension(CoverageClass coverage) { … }` declares members that read as if `CoverageClass` had them: methods and properties, instance or static, and user-defined operators. There are no fields — extensions still hold no state (What's new in C# 14 · extension member declarations). Extension *indexers* arrive with the C# 15 preview (checked 2026-09-20) **• Verified**.

For enums this is where behaviour goes. Kotlin puts code and `fromCode` inside the `enum class`; a C# enum cannot hold members, so they live in an extension block. Classic `this`-parameter extension methods (panel 6.2) compile to the same IL, so

existing libraries can move to the new syntax without breaking callers.

`==` means whatever the type says it means. On a class it compares references unless the class overloads it — the opposite of Kotlin, where `==` calls `equals` (• Trap). Records and `string` overload it. A plain struct has no `==` at all until you declare one, and its default `Equals` compares fields through reflection. Two boxed `5`s typed as `object` are not `==` (equality comparisons).

`Equals` and `GetHashCode` are one contract — the same one as Java's. Compare VINs case-insensitively but hash them case-sensitively and a `HashSet` loses them: `L03.TypesTour` prints `False` for exactly that bug. Operators are static methods with reserved names (`op_Addition`, `op_Equality`); define `==` and `!=` as a pair, and Visual Basic calls them like its own.

6.1 · Behaviour for CoverageClass — Kotlin enum class vs a C# 14 extension block

Kotlin — for comparison — the idea you already know

```kotlin
// Kotlin: an enum is a class and holds members
enum class CoverageClass(val code: String) {
    CLASS_1("1"), CLASS_2_PLUS("2+"),
    CLASS_3_PLUS("3+"), CLASS_3("3");

    companion object {
        fun fromCode(code: String) =
            entries.first { it.code == code }
    }
}

// ...and extensions add more from outside
val CoverageClass.coversOwnDamage: Boolean
    get() = this != CoverageClass.CLASS_3

// CoverageClass.CLASS_2_PLUS.code       -> "2+"
// CoverageClass.fromCode("3+")          -> CLASS_3_PLUS
```

C# — samples/L03.Domain/Extensions.cs · #region extension-block

```csharp
public static class CoverageClassExtensions
{
    // C# 14: members for a type you cannot change
    extension(CoverageClass coverage)
    {
        public bool CoversOwnDamage => coverage
            is CoverageClass.Class1
            or CoverageClass.Class2Plus
            or CoverageClass.Class3Plus;

        public string Code => coverage switch
        {
            CoverageClass.Class1 => "1",
            CoverageClass.Class2Plus => "2+",
            CoverageClass.Class3Plus => "3+",
            CoverageClass.Class3 => "3",
            _ => throw new ArgumentOutOfRangeException(
                nameof(coverage)), // enums are open
        };
    }

    // no receiver name: static members of CoverageClass
    extension(CoverageClass)
    {
        public static CoverageClass FromCode(string code) =>
            Enum.GetValues<CoverageClass>()
                .First(c => c.Code == code);
    }
}
```

Same call sites: `CoverageClass.Class2Plus.Code` and `CoverageClass.FromCode("3+")`. The first block names its receiver (`coverage`) and holds instance members; the second has no name and holds static members that appear on the enum itself. The default arm is not decoration: an enum is an integer, and `(CoverageClass)42` reaches it.

6.2 · A classic extension method — the form LINQ and services.AddX() use

C# — samples/L03.Domain/Extensions.cs · #region classic-extension

```csharp
public static class DriverExtensions
{
    // the pre-C# 14 form: a static method with a `this` parameter
    public static bool IsYoungOn(this Driver driver, DateOnly day) =>
        driver.AgeOn(day) < 25;
}
```

• Same idea **idea as a Kotlin extension function, and the form most .NET code still uses.** A `static` method in a `static` class whose first parameter carries `this` reads as `driver.IsYoungOn(day)`; every LINQ operator and every `services.AddX()` registration call is declared this way. It is visible only where its namespace is imported, and it can add methods only — properties, static members and operators need the C# 14 block of panel 6.1. `ExtensionTests` calls both forms.

6.3 · Case-insensitive VIN equality — by hand, and as a record

C# samples/L03.Domain/Vin.cs · #region equatable-class

```csharp
// the pre-records way: every equality member by hand
public sealed class Vin : IEquatable<Vin>
{
    private static readonly StringComparer Rule =
        StringComparer.OrdinalIgnoreCase;

    public Vin(string value) => Value = value;
    public string Value { get; }

    public bool Equals(Vin? other) =>
        other is not null && Rule.Equals(Value, other.Value);
    public override bool Equals(object? obj) =>
        Equals(obj as Vin);
    public override int GetHashCode() =>
        Rule.GetHashCode(Value); // the SAME rule as Equals
    public static bool operator ==(Vin? a, Vin? b) =>
        a is null ? b is null : a.Equals(b);
    public static bool operator !=(Vin? a, Vin? b) =>
        !(a == b);
}
```

C# samples/L03.Domain/Vin.cs · #region equatable-record

```csharp
// the same contract as a record: replace only what differs
public sealed record VinRecord(string Value)
{
    private static readonly StringComparer Rule =
        StringComparer.OrdinalIgnoreCase;

    public bool Equals(VinRecord? other) =>
        other is not null && Rule.Equals(Value, other.Value);

    public override int GetHashCode() =>
        Rule.GetHashCode(Value);

    // Equals(object), == and != are still generated,
    // and they call the Equals above
}
```

The left is what legacy .NET code looks like; the right is what to write now. Both compare and hash with the same `OrdinalIgnoreCase` rule. A record lets you replace just `Equals(VinRecord?)` and `GetHashCode`; the generated `Equals(object)`, `==` and `!=` route through your version. `EqualityTests` checks both types in a `HashSet`.

6.4 · Boxes, overloaded == and a broken hash code

C# samples/L03.TypesTour/Program.cs · #region boxing-equality

```csharp
object five = 5, alsoFive = 5;
Console.WriteLine(five == alsoFive);        // False: two boxes
Console.WriteLine(five.Equals(alsoFive));   // True: Int32.Equals

var vinA = new Vin("MR0FZ22G801234567");
var vinB = new Vin("mr0fz22g801234567");
Console.WriteLine(vinA == vinB);            // True: overloaded ==
Console.WriteLine(new HashSet<Vin> { vinA }.Contains(vinB)); // True

var broken = new HashSet<BrokenVin> { new("MR0FZ22G801234567") };
Console.WriteLine(broken.Contains(new("mr0fz22g801234567"))); // False
```

Output, in order: `False`, `True`, `True`, `True`, `False` (captured, §8). Two boxes of 5 are different objects, so `==` on `object` is false while `Equals` is true. `BrokenVin` is equal case-insensitively but hashes case-sensitively, so the set looks in the wrong bucket and never finds the lower-case VIN.

7 · Design in C# — a multi-format report framework, extended from VB

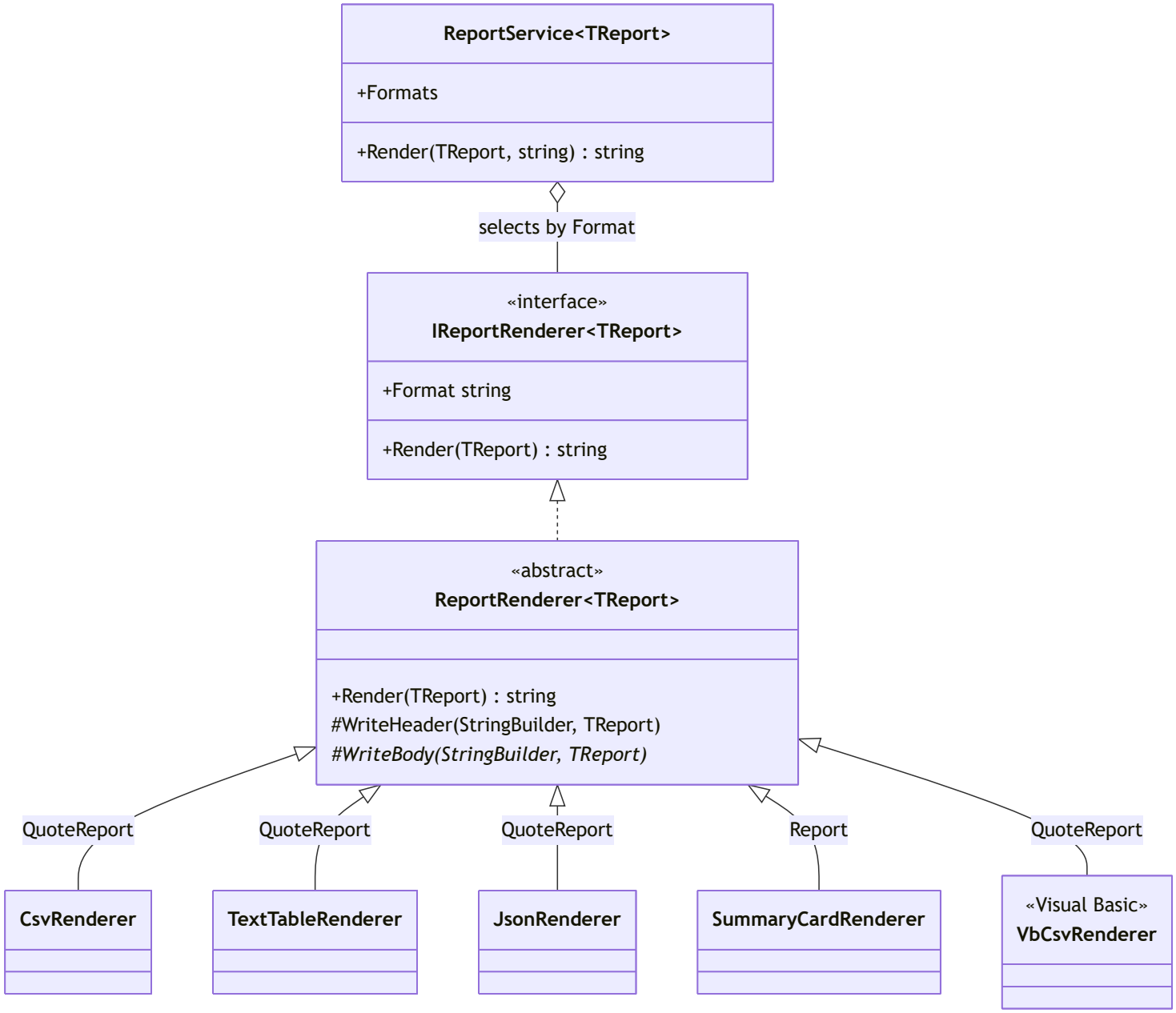
This is the multi-format report framework you have built before, with the type system doing more of the work. `IReportRenderer<in TReport>` is the strategy; `ReportRenderer<TReport>` is a template-method base class whose non-virtual `Render` fixes the skeleton; `ReportService<TReport>` picks a renderer by format name. The interface is the contract callers and the container see; the abstract class is optional shared skeleton an implementer may inherit — keep both, so a renderer that shares nothing can implement the interface directly. The sample renders text, CSV and JSON strings where production code would write PDF, Excel and JSON files — the shape does not change.

SOLID transfers as-is (• Same idea); the .NET-specific parts are variance and the constructor shape. A new format is a new class and no edits (open/closed). The service depends on `IEnumerable<IReportRenderer<TReport>>`, which the ASP.NET Core container fills with every registration of exactly that closed type, as Spring fills a `List<ReportRenderer>` — registration is lesson 08 (ASP.NET Core Web APIs). Contravariance lets one `SummaryCardRenderer` serve every report type when you assign it, but the built-in container does not apply variance: register it under each `IReportRenderer<X>` you need. Because `Render` is not virtual, no subclass in either language can reorder or skip the header hook when called through `ReportRenderer<TReport>` — short of re-implementing the interface, which a code review should reject.

Visual Basic can extend the framework, but it cannot declare the modern types it consumes. `L03.VbInterop` inherits the C# generic base class, overrides its hooks, and produces CSV identical to the C# renderer's (`same CSV from C# and VB? True`). It constructs and compares C# records, uses their operators, and sets `required` and `init` members in a `With { }` initializer — Visual Basic 16.9 added init-only consumption (What's new for Visual Basic, checked 2026-09-20) • Verified. It has no syntax to declare any of them, in line with Microsoft's consumption-only strategy for VB (Visual Basic language strategy); the migration plan that follows from it is lesson 06 (VB.NET for Legacy Estates).

Pin the language boundary with a test, not a wiki page. In a mixed estate the risk is a C# change that quietly makes a shared type unusable from VB. `VbCompilerTests` compiles Visual Basic snippets against the C# assemblies with Roslyn's VB compiler (package 5.9.0, the compiler line SDK 10.0.401 ships) and asserts the exact error id of every limit in table 7.6 (panel 7.7), and that the call-site workarounds compile.

7.1 · The report framework — one C# contract, renderers in two languages



interface and abstract base in L03.Reports · VbCsvRenderer lives in the Visual Basic project · SummaryCardRenderer renders any Report and fits QuoteReport through contravariance

7.2 · The template-method base class

C# — samples/L03.Reports/Renderers.cs · #region renderer-base

```
public abstract class ReportRenderer<TReport> : IReportRenderer<TReport>
    where TReport : Report
{
    public abstract string Format { get; }

    // not virtual: the skeleton is fixed (template method)
    public string Render(TReport report)
    {
        var text = new StringBuilder();
        WriteHeader(text, report);
        WriteBody(text, report);
        return text.ToString().TrimEnd();
```

```
    }

    // virtual: a hook with a default a subclass may replace
    protected virtual void WriteHeader(StringBuilder text, TReport report) =>
        text.AppendLine(Invariant($"# {report.Title} - {report.AsOf:yyyy-MM-dd}"));

    // abstract: every renderer must supply the body
    protected abstract void WriteBody(StringBuilder text, TReport report);
}
```

Three kinds of member, three promises. `Render` is not virtual: every renderer runs header, then body. `WriteHeader` is `virtual` with a default — CSV replaces it, JSON empties it. `WriteBody` is `abstract`. `Invariant($"…")` formats the date without the machine's culture, so a Thai-locale server does not print a Buddhist-calendar year.

7.3 · The strategy selector — DI-friendly and case-insensitive

C# — samples/L03.Reports/Renderers.cs · #region strategy

```
// DI-friendly: the container passes every IReportRenderer<TReport> registration
public sealed class ReportService<TReport>(
    IEnumerable<IReportRenderer<TReport>> renderers)
    where TReport : Report
{
    private readonly Dictionary<string, IReportRenderer<TReport>>
        _byFormat = renderers.ToDictionary(
            r => r.Format, StringComparer.OrdinalIgnoreCase);

    public IEnumerable<string> Formats => _byFormat.Keys;

    public string Render(TReport report, string format) =>
        _byFormat.TryGetValue(format, out var renderer)
            ? renderer.Render(report)
            : throw new NotSupportedException($"format '{format}'");
}
```

The constructor is the whole integration surface. Tests pass renderers as a collection expression; a DI container passes every registration of exactly `IReportRenderer<TReport>`. The primary-constructor parameter is used only by the field initializer, so it is never stored. An unknown format throws `NotSupportedException` rather than returning an empty report.

7.4 · The same CSV renderer in C# and in Visual Basic

C# — samples/L03.Reports/Renderers.cs · #region csv-renderer

```
// the "Excel" format: one CSV row per quote line
public sealed class CsvRenderer : ReportRenderer<QuoteReport>
{
    private static readonly CultureInfo Inv =
        CultureInfo.InvariantCulture;

    public override string Format => "csv";

    protected override void WriteHeader(
        StringBuilder text, QuoteReport report) =>
        text.AppendLine("quote,vehicle,class,total_thb");

    protected override void WriteBody(
        StringBuilder text, QuoteReport report)
    {
        foreach (var q in report.Lines)
            text.AppendLine(string.Join(",",
                q.Id, q.Vehicle, q.Coverage.Code,
                q.Total.Amount.ToString(Inv)));
    }
}
```

Visual Basic — samples/L03.VbInterop/Program.vb · #region vb-csv

```
' the same CSV renderer in VB, on the C# generic base class
Public NotInheritable Class VbCsvRenderer
    Inherits ReportRenderer(Of QuoteReport)

    Private Shared ReadOnly Inv As CultureInfo =
        CultureInfo.InvariantCulture

    Public Overrides ReadOnly Property Format As String = "vb"

    Protected Overrides Sub WriteHeader(
            text As StringBuilder, report As QuoteReport)
        text.AppendLine("quote,vehicle,class,total_thb")
    End Sub

    Protected Overrides Sub WriteBody(
            text As StringBuilder, report As QuoteReport)
        For Each q In report.Lines
            text.AppendLine(String.Join(",",
                q.Id, q.Vehicle,
                CoverageClassExtensions.get_Code(q.Coverage),
                q.Total.Amount.ToString(Inv)))
        Next
    End Sub
End Class
```

One base class, two languages, byte-identical output. VB writes `Inherits ReportRenderer(Of QuoteReport)` for `: ReportRenderer<QuoteReport>` and `Protected Overrides Sub` for `protected override void`. The one real difference is the class code: VB calls the extension property's generated `get_Code(…)` method (table 7.6, row 1). VB is longer, not harder: every member and loop ends on its own line (`End Sub`, `Next`) where C# uses braces or an expression body.

7.5 · What Visual Basic does with C# records, operators and required members

Visual Basic — samples/L03.VbInterop/Program.vb · #region vb-consume

```
' C# records: construct, read and compare them
Dim young = Samples.YoungDriver()
Dim sameRequest = Samples.YoungDriver()
Console.WriteLine($"records    equal? {young = sameRequest}")

' no `with` in VB: build the changed copy by hand
Dim class3 = New QuoteRequest(young.Vehicle, young.Driver,
                              CoverageClass.Class3, young.StartDate)
Console.WriteLine($"copy       equal? {young = class3}")

' C# operators on a record struct
Dim sum = Money.Thb(1000D) + Money.Thb(500D)
Console.WriteLine($"operators  {sum}, same? {sum = Money.Thb(1500D)}")

' required + init members: set them inside With { } only
Dim quote As New Quote With {
    .Id = New QuoteId(7),
    .Request = young,
    .Premium = Samples.Calculator().Calculate(young)}
Console.WriteLine($"required   {quote.Id} total {quote.Premium.Total}")

' a C# 14 extension property has no VB syntax: call its method
Dim code = CoverageClassExtensions.get_Code(CoverageClass.Class2Plus)
Console.WriteLine($"extension  get_Code(Class2Plus) = {code}")
```

Consumption works; declaration does not exist. VB compares records with =, because C# emitted `op_Equality`. It has no `with`, so the changed copy is built by calling the constructor. VB also *enforces* the C# contract on `required` and `init` members — the compiler errors are in table 7.6.

7.6 · What the Visual Basic compiler says — every row asserted by VbCompilerTests

C# feature	Written in Visual Basic	VB compiler	What works instead
Extension property · C# 14	`CoverageClass.Class2Plus.Code`	`BC30456` not a member	`CoverageClassExtensions.get_Code(…)`
Static extension member · C# 14	`CoverageClass.FromCode("3+")`	`BC30456` not a member	`CoverageClassExtensions.FromCode("3+")`
`required` member · C# 11	`New Quote With {…}` without `.Premium`	`BC37321` must be set	Set every required member in `With { }` — VB enforces the contract
`init` accessor · C# 9	`Q.ValidUntil = R.StartDate`	`BC37311` init-only	Assign it inside `With { }`
Default interface member · C# 8	`New StandardRateTable().BasePremium(R)`	`BC30456` not a member	Call it through an `IRateTable` reference
Default interface member · C# 8	A VB class implementing `IRateTable` with only `RateFor`	`BC30149` must implement	Write `BasePremium` in the VB class
`with` expression · C# 9	`R With {.Coverage = …}`	`BC30205` syntax error	Call the record's constructor
Static abstract member · C# 11	A VB `Structure` implementing `IIdentifier(Of T)`	`BC37315` cannot implement	Declare such types in C#
Static abstract member · C# 11	`T.Parse(text)` inside a VB generic method	`BC32098` type parameter as qualifier	Call a C# generic helper: `Ids.ParseAll(Of T)`

7.7 · Pinning the boundary — Visual Basic compiled inside an xUnit test

C# — samples/L03.Reports.Tests/VbCompilerTests.cs · #region vb-compiler-tests

```
// each snippet runs inside: Sub Run(R As QuoteRequest, Q As Quote)
[Theory]
[InlineData("Dim c = CoverageClass.Class2Plus.Code", "BC30456")]
[InlineData("Dim f = CoverageClass.FromCode(\"3+\")", "BC30456")]
[InlineData("Dim quote As New Quote With {.Id = New QuoteId(1), .Request = R}", "BC37321")]
[InlineData("Q.ValidUntil = R.StartDate", "BC37311")]
[InlineData("Dim p = New StandardRateTable().BasePremium(R)", "BC30456")]
[InlineData("Dim r2 = R With {.Coverage = CoverageClass.Class3}", "BC30205")]
public void A_CSharp_only_feature_fails_to_compile(string vb, string error) =>
    Assert.Equal([error], Errors(InSub(vb)));

// ...and the workaround for each compiles cleanly
[Theory]
[InlineData("Dim c = CoverageClassExtensions.get_Code(CoverageClass.Class2Plus)")]
[InlineData("Dim f = CoverageClassExtensions.FromCode(\"3+\")")]
[InlineData("Dim p = CType(New StandardRateTable(), IRateTable).BasePremium(R)")]
[InlineData("Dim r2 = New QuoteRequest(R.Vehicle, R.Driver, CoverageClass.Class3, R.StartDate)")]
public void Its_workaround_compiles(string vb) =>
    Assert.Empty(Errors(InSub(vb)));
```

Table 7.6 is test data, not recollection. Each row compiles one VB statement with `Option Strict On` against `L03.Domain` and asserts the complete set of error ids, so a snippet that fails for any other reason fails the test. The declaration rows sit in a second theory in the same file, beside one more VB fact: `Dim q` in a method with a parameter `Q` is BC30734, because VB names are case-insensitive — see lesson 06 (VB.NET for Legacy Estates).

7.8 · Declare, consume or work around — Visual Basic on modern C# types

Visual Basic declares

- Classes, structures and interfaces, with `In` / `Out` variance
- `MustInherit` · `Overridable` · `Shadows` hierarchies
- Operators and classic `<Extension>` methods
- Generic types and methods: `(Of T As IIdentifier(Of T))`

Visual Basic only consumes

- Records and record structs — construct, compare, no `with`
- `init` and `required` members, inside `With { }`
- Default interface methods, through the interface
- Generic methods constrained on static abstract members

Visual Basic cannot reach as syntax

- C# 14 extension properties — call `get_Code(…)` instead
- Static extension members on the extended type
- `with` expressions
- Calling `T.Parse` on its own type parameters

8 · Hands-on — build, test and run the lesson-03 samples

Six projects, one command to prove them. `verify_samples.py --only 3` builds the two libraries, runs both xUnit projects, and runs both consoles to completion. Every C# and VB panel in this PDF was cut from those files, and every byte count and method count came from the `L03.TypesTour` output below.

Read the two outputs against the sections. Lines 2 and 3 of the tour are panel 2.4; `alloc` and `generated` are charts 2.5 and 2.6; the three bare lines after `dispatch` are panel 4.2; the five after `equality` are panel 6.4. The VB output proves §4 and §7: VB's own hierarchy dispatched like C#'s, and VB compared C# records, set required members, called an extension property through its method and rendered the same CSV. What VB *cannot* do is proved by `VbCompilerTests` inside `L03.Reports.Tests` (7.7).

Then break something on purpose. Remove `virtual` from `RatingRule.Factor` and watch the build fail on CS0506; delete `Premium = …` from a `new Quote { … }` and read CS9035; make `BrokenVin` hash case-insensitively and watch `EqualityTests` fail its `False` assertion.

8.1 · Commands

Terminal

```
# build, test and run all six lesson-03 projects
python 0-script/verify_samples.py --only 3

# the measurements quoted in this lesson
dotnet run --project lesson-03-types-oop-generics/samples/L03.TypesTour -c Release

# Visual Basic extending the C# report framework
dotnet run --project lesson-03-types-oop-generics/samples/L03.VbInterop -c Release

# the tests on their own
dotnet test lesson-03-types-oop-generics/samples/L03.Domain.Tests
dotnet test lesson-03-types-oop-generics/samples/L03.Reports.Tests
```

> **`verify_samples.py` is the proof; the two `dotnet run` lines reproduce panels 8.2 and 8.3.** Run everything from the repository root, with SDK 10.0.401 or a later 10.0 feature band (`global.json`). `dotnet run` and `dotnet test` restore and build what they need, so there is no separate restore step.

8.2 · What you should see — the C# tour

Output — L03.TypesTour captured on the build machine

```
L03.TypesTour on .NET 10.0.12
copy        copy.Count=1 list[0].Count=0
default     default(Money).Currency is null: True
size        Money=24 bytes, QuoteId=4, CoverageClass=4
alloc       Money[]=24,024 MoneyClass[]=48,024 object[] of Money=48,024 bytes
generated   class=3 struct=3 record=16 record struct=13 methods
dispatch    asBase.Factor / asBase.Describe / asDerived.Describe:
1.20
young driver x1.20
young driver (hidden copy)
dim         IRateTable.BasePremium = 9,450.00 THB
reified     QuoteId Q- Q-000000 | PolicyNumber P- P-00000000 | parsed Q-000042, Q-000043
generics    List<int>=4,056 List<object>=32,056 ArrayList=32,056 bytes for 1,000 ints
statics     Counter<QuoteId>.Hits=2 Counter<PolicyNumber>.Hits=1
math        Sum<decimal>=6.50 Sum<Money>=1,500.00 THB
variance    card renderer as IReportRenderer<QuoteReport>: # Motor quotes - 2026-10-01 (QuoteReport)
extension   Class2Plus.Code=2+ Class3.CoversOwnDamage=False FromCode("3+")=Class3Plus
enum        (CoverageClass)42 is defined: False
equality    five==alsoFive / Equals / vinA==vinB / set.Contains / broken.Contains:
False
True
True
True
False
premium     net 7,938.00 THB, stamp duty 31.75 THB, VAT 557.88 THB, total 8,527.63 THB
status      Q-000042 Accepted -> policy P-00000007; Expire() now throws (Accepted -> Expired)
```

> **Only the first line should differ on your machine** — the runtime patch number. The byte counts assume a 64-bit process; the rest is deterministic because the tour fixes the culture and uses no clock or random numbers.

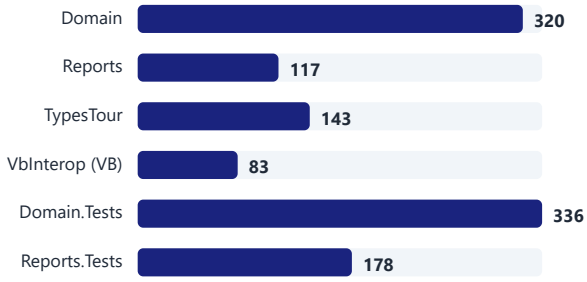
8.3 · What you should see — Visual Basic on the C# types

Output — L03.VbInterop captured on the build machine

```
L03.VbInterop - Visual Basic using the lesson-03 C# types
records    equal? True
copy       equal? False
operators  1,500.00 THB, same? True
required   Q-000007 total 8,527.63 THB
extension  get_Code(Class2Plus) = 2+
dispatch   1.20 | young driver x1.20 | young driver (hidden copy)
generics   Q-000042, Q-000043
same CSV from C# and VB? True
# Motor quotes - 2026-10-01
Q-000042  Toyota Yaris   class 1         8,527.63 THB
Q-000043  Isuzu D-Max    class 2+       10,790.07 THB
2 quotes, total 19,317.70 THB
```

The key line is `same CSV from C# and VB? True`. `dispatch 1.20 | young driver x1.20 | young driver (hidden copy)` is panel 4.2 again, from the VB hierarchy of panel 4.7: `Overridable` follows the object, `Shadows` follows the declared type.

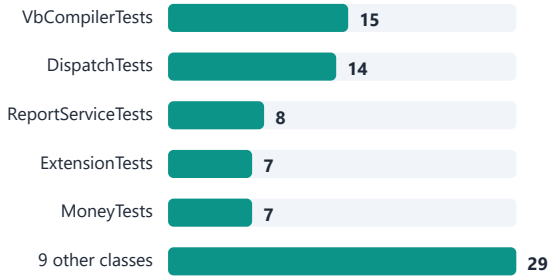
8.4 · Code lines per sample project



non-blank, non-comment lines of .cs and .vb files · measured at build

The two test projects hold 514 lines against 437 in the two libraries they test. Line counts are not coverage: nothing in this lesson measures which lines the tests execute. lesson 10 (Testing, TDD & 100% Coverage) measures coverage and makes it a gate.

8.5 · Test cases per test class



[Fact] methods plus [InlineData] rows · the smallest classes combined · measured at build

Every item in the Traps callout below has a test that fails if the trap is removed — except exhaustiveness over a class hierarchy, which C# 14 cannot express. `VbCompilerTests` is the largest class (15 cases): the boundary with Visual Basic is where a C# change breaks code the C# team never compiles.

⚠ Traps — they look right to a JVM engineer

1. **Overriding without `virtual` and `override`.** A same-named method hides the base method (CS0114 when the base is virtual, CS0108 when it is not); calls through the base type still run the base code. Here the warning is an error; in many legacy solutions it is not.
2. **`==` on a class or on `object` compares references.** Two boxed `5`s are not `==`. Use `Equals`, a record, or an overloaded operator.
3. **Structs are copied.** A struct read from a `List<T>` or a property is a copy, and `default(Money)` or `new Money[n]` runs no constructor — `Currency` is null even though its type says it cannot be.
4. **A class's primary-constructor parameters are not `val`s.** They are captured, mutable and invisible as properties; copy them into `readonly` fields when that matters.
5. **Enums are open integers and `sealed` means final.** Neither gives you Kotlin's exhaustive `when`. Keep a default arm on class-hierarchy switches until `closed` (C# 15 preview) ships; enum switches always need one.

✓ Checkpoint — you are ready for lesson 04 when

1. `python 0-script/verify_samples.py --only 3` reports 6/6 sample projects passed.
2. You can place every MotorQuote type on diagram 2.7 and defend the choice in terms of chart 2.5.
3. You can predict the three lines of panel 4.2 before running the tour, and say which keyword change would alter each.
4. You can explain why `SummaryCardRenderer` fits `IReportRenderer<QuoteReport>` while a `List<QuoteReport>` never fits `List<Report>`.
5. You can name the two C# 14 features Visual Basic can only reach through a workaround, and three older C# features it can consume but not declare.

🔍 Check yourself

1. How many bytes did 1,000 `Money` values take as a `Money[]`, as class instances and as boxes — and why is a box no cheaper than a class?
2. `RatingRule rule = new YoungDriverLoading();` — which `Describe` runs, and why does its text still contain the derived factor 1.20?
3. What does `default(Money).Currency` return, and which constructor ran to produce it?
4. Why is an `IReportRenderer<Report>` assignable to `IReportRenderer<QuoteReport>`, while `IEnumerable<int>` is not assignable to `IEnumerable<object>`?
5. Which member kinds can a C# 14 extension block declare, which arrive in C# 15, and how does Visual Basic call the `Code` extension property?
6. Why is `ReportRenderer<TReport>.Render` deliberately not virtual, and which call path could a subclass still use to skip the header?
7. In `PremiumCalculator(IRateTable rates, IEnumerable<RatingRule> rules)`, which parameter does the compiler store, can the class reassign it, and why does the class copy `rules` into a `readonly` array?

Lesson 03 in one line: choose reference or value semantics per type — **class** for entities, **readonly record struct** for values, **record** for data — remember that methods are non-virtual and generics are reified, and let Visual Basic consume what it cannot declare.
Next: lesson 04 (Collections, LINQ & Functional C#) — collections, LINQ and the functional side of C#.